Easool mobile app — complete guide for new developers

This document explains **what the Easool app is**, **how files fit together**, **what happens from app launch to login**, **how every API call works**, and **how authentication flows** from the phone to the backend. You do not need prior mobile experience to follow it; technical terms are explained as we go.

Table of contents

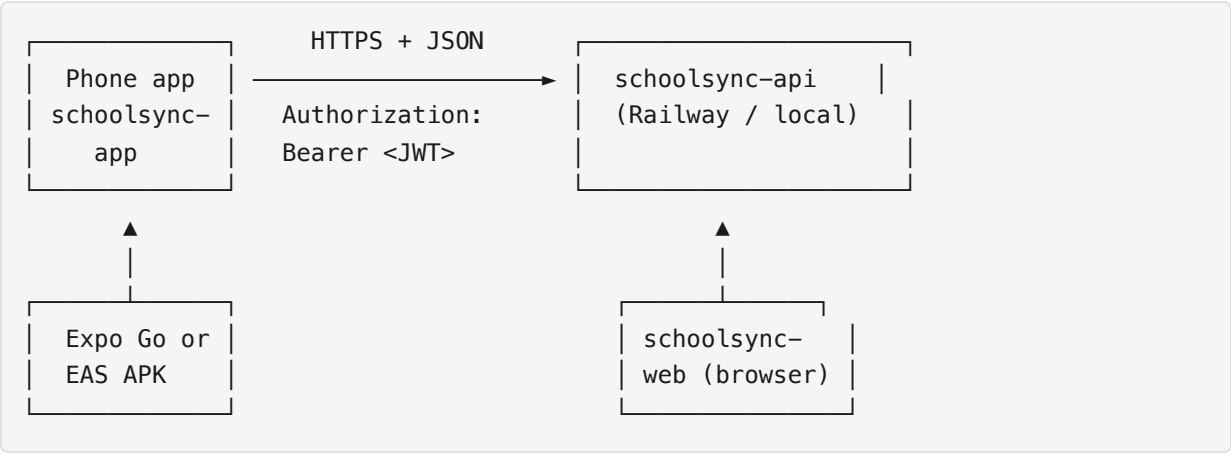
1. The big picture
2. Basic terms (JSON, TypeScript, files)
3. Technology stack
4. Repository folders
5. How the app starts and runs
6. Navigation: how screens are chosen
7. Authentication (end to end)
8. How API calls work
9. JSON the app sends and receives
10. Roles and what users see
11. Walkthrough: login then home dashboard
12. Walkthrough: any feature screen
13. Configuration, builds, and environments
14. Where the website fits in
15. Glossary
16. What to read next in the code

---

1. The big picture

Easool is school operations software (attendance, timetables, notices, approvals, etc.). There are **three separate codebases** you should know about:

Piece	Repo / folder	What it does
Backend API	`schoolsync` (Java Spring Boot)	Stores data, enforces rules, issues JWT tokens. Hosted on Railway in production.
Website	`schoolsync-web` (React)	easool.com — same API, runs in the browser.
Mobile app	`schoolsync-app` (this repo)	Android/iOS app built with **Expo + React Native**. **No server of its own** — only talks to the backend API.



The mobile app is **not a copy of the website UI** (different layout — tabs, native scroll, etc.), but it uses the **same login**, **same JWT**, and **same REST endpoints** as the web.

---

2. Basic terms (JSON, TypeScript, files)

Term	Meaning
JSON	Text format for data over the network. Example: `{ "phone": "QA-SCH-001-STU-0001", "otp": "secret" }`. Keys are in quotes; the API almost always wraps answers in `{ "success", "message", "data" }`.
TypeScript (`.ts`)	JavaScript with types (`userId: string`) so mistakes are caught before build.
TSX (`.tsx`)	TypeScript + UI markup. A screen file like `login.tsx` defines React components (buttons, text fields).
React	UI library: you describe screens as components; when `user` state changes, the screen re-renders.
React Native	React for phones: `<Text>`, `<View>` instead of HTML `<div>`.

Term	Meaning
Expo	Toolkit on top of React Native: dev server, native modules (secure storage), cloud builds (EAS).
Expo Router	**File-based routing**: file path under `app/` = URL path. `app/login.tsx` → `/login`.
JWT / token	A signed string from the server proving "this request is user X". The app stores it and sends `Authorization: Bearer <token>` on protected calls.
Axios	HTTP client library used in `src/api/client.ts` to POST/GET the API.
SecureStore	Encrypted storage on the device for the JWT (via `expo-secure-store`).

---

3. Technology stack

Layer	Choice in this repo
Runtime on phone	React Native **0.79** (Expo SDK **53**)
Language	**TypeScript**
Routing	**expo-router** (`app/` folder)
HTTP	**axios**
Server state (optional)	**@tanstack/react-query** (`QueryClient` in root layout; many screens use simple `useState` + `useLoad` instead)
Auth session	**React Context** (`src/auth/AuthContext.tsx`) + **SecureStore**

Important files at repo root:

File	Purpose
`package.json`	NPM dependencies and scripts (`npm start`, `npm run typecheck`).
`app.json`	Expo app name, icon, Android package id, EAS `projectId`.
`eas.json`	Cloud build profiles (`preview` = APK for testers, `production` = Play Store AAB).
`.env` (not committed)	`EXPO_PUBLIC_API_BASE_URL` — which API host the built app uses.

File	Purpose
`.env.example`	Template for `.env`.

---

4. Repository folders

```
schoolsync-app/
├── app/                    # Screens (Expo Router) — one file ≈ one route
│   ├── _layout.tsx         # Root: providers, auth gate, stack navigator
│   ├── index.tsx           # Cold start: redirect to contact or tabs
│   ├── login.tsx           # Sign in
│   ├── contact.tsx         # Guest landing / "contact us"
│   ├── (tabs)/             # Bottom tab area (signed-in shell)
│   │   ├── _layout.tsx     # Tab bar: Home, Notices, Alerts, Menu
│   │   ├── index.tsx       # Home dashboard
│   │   ├── notices.tsx
│   │   ├── notifications.tsx
│   │   └── menu.tsx
│   ├── attendance/         # e.g. students.tsx, teachers.tsx
│   ├── super-admin/        # Platform operator screens
│   ├── timetable/
│   └── …                   # approvals, records, directory, etc.
├── src/
│   ├── api/                # One module per backend area (thin wrappers)
│   │   ├── client.ts       # Axios instance + JWT interceptor + 401 logout
│   │   ├── auth.ts         # login, register, select-student, …
│   │   ├── loginAuth.ts    # Parse login JSON into app-shaped auth object
│   │   ├── dashboard.ts
│   │   └── …
│   ├── auth/
│   │   └── AuthContext.tsx # Global user session
│   ├── ui/index.tsx        # Shared buttons, fields, Screen, Card, …
│   ├── menu.ts             # Role-based menu items (mirrors web nav)
│   ├── theme.ts            # Colors (Easool light theme)
│   ├── config.ts           # API base URL from env
│   └── components/         # BrandLogo, NoticeStrip, ErrorBoundary, …
├── assets/                 # logo.png, images
├── APP_GUIDE.md            # This file
├── README.md               # Short setup notes
└── PILOT_RUNBOOK.md        # EAS / APK testing without a Mac
```

Rule of thumb: If you want to change **what the user sees**, start in `app/…`. If you want to change **how data is fetched**, start in `src/api/…`.

---

5. How the app starts and runs

5.1 Entry point

Expo Router's entry is configured in `package.json`:

```
"main": "expo-router/entry"
```

That bootstrap loads **`app/_layout.tsx`** first (the root layout).

5.2 Root layout — wrappers and "Gate"

File: `app/_layout.tsx`

Execution order (outside → inside):

1. **`SafeAreaProvider`** — respects notches / status bar / home indicator.
2. **`QueryClientProvider`** — React Query (global cache; used lightly).
3. **`AuthProvider`** — loads saved token/user from SecureStore on launch.
4. **`Gate`** — while `loading`, shows spinner; when signed out, keeps user on `contact` / `login`; when signed in, renders the navigation stack.
5. **`ErrorBoundary`** — catches render crashes and shows a friendly error instead of closing the app.

5.3 Cold start route

File: `app/index.tsx`

```
App opens
    → AuthProvider reads SecureStore (token + user JSON)
    → index.tsx runs
         if still loading → spinner
         if user exists  → <Redirect href="/(tabs)" />
         if no user      → <Redirect href="/contact" />
```

Guests see **Contact**; returning users skip login and land on **tabs**.

5.4 Sequence diagram (launch → signed-in home)

> **Diagram (flow chart):** The full interactive version is in `APP_GUIDE.md` under the same section. Summary:

```
sequenceDiagram
    participant OS as Android/iOS
    participant RN as React Native bundle
    participant Layout as app/_layout.tsx
    participant Auth as AuthContext
    participant Store as Secur…
```

---

6. Navigation: how screens are chosen

Expo Router maps **file paths to URLs**:

File	Route (approx.)	Who sees it
`app/contact.tsx`	`/contact`	Guests
`app/login.tsx`	`/login`	Sign-in
`app/(tabs)/index.tsx`	`/(tabs)` home	Signed-in
`app/approvals.tsx`	`/approvals`	Stack screen from Menu / Home links
`app/attendance/students.tsx`	`/attendance/students`	Teachers, principals, …

- **`(tabs)`** — parentheses mean a **route group** (bottom tab bar), not a visible URL segment.
- **Stack screens** — most `app/*.tsx` files outside `(tabs)` push on top of tabs when you `router.push("/approvals")`.

Menu tab (`app/(tabs)/menu.tsx`) reads `src/menu.ts` and shows only items allowed for `user.role`.

---

7. Authentication (end to end)

7.1 Concepts

Concept	Where it lives
Login request	`app/login.tsx` → `src/api/auth.ts` → `POST /auth/login`
Parse response	`src/api/loginAuth.ts` → `extractLoginPayload()`
Save session	`src/auth/AuthContext.tsx` → `applySession()` → SecureStore

Concept	Where it lives
Attach token to requests	`src/api/client.ts` request interceptor
Expired / invalid token	Response `401` → `onUnauthorized()` → `logout()` clears store
Current user in UI	`useAuth()` hook anywhere in the tree under `AuthProvider`

7.2 What the user types

The app login field is labeled **UserName** (school-issued username, e.g. `QA-SCH-001-STU-0001`). It is sent as `phone` in the JSON body because the backend `LoginRequest` field is still named `phone` (historical name — it accepts username too).

7.3 Login steps (password)

> **Diagram (flow chart):** The full interactive version is in `APP_GUIDE.md` under the same section. Summary:

```
sequenceDiagram
    participant User
    participant Login as app/login.tsx
    participant AuthAPI as src/api/auth.ts
    participant Backend as POST /auth/login
    participant Parse as loginAuth.ts…
```

7.4 What is stored on the device

After login, **two keys** in SecureStore:

Key	Content
`ss_token`	JWT string
`ss_user`	JSON string: `{ userId, schoolId, role, name, schoolName? }`

On next app open, `AuthProvider` reads these **without** calling the API again (until token expires or API returns 401).

If `schoolName` was missing, `AuthContext` may call `GET /profile` once to fill it in (`hydrateSchoolName`).

7.5 Public vs protected routes

Public (no token required): e.g. `POST /auth/login`, `GET /auth/schools`, `POST /auth/register`.

Protected (token required): almost everything else (`/dashboard/stats`, `/notices`, `/attendance/...`). The backend Spring Security validates the JWT; the app only adds the header.

---

8. How API calls work

8.1 Central client

File: **`src/api/client.ts`**

```
1. axios.create({ baseURL: API_BASE_URL, timeout: 20000 })
2. Before each request:
      token = await SecureStore.getItemAsync("ss_token")
      if token → header Authorization: Bearer <token>
3. On response error with status 401:
      call registered logout handler (clear session)
```

`API_BASE_URL` comes from **`src/config.ts`**:

- Default production: `https://schoolsync-api-production.up.railway.app`
- Override at build time: `EXPO_PUBLIC_API_BASE_URL` in `.env` or `eas.json` `env` block

Important: Unlike the website dev server, the app **does not** use a `/api` prefix. It hits the API host directly (`/auth/login`, not `/api/auth/login`).

8.2 Per-feature modules

Each file under `src/api/` is a **thin wrapper**:

```
// src/api/dashboard.ts
import api from "./client";
export const getDashboardStats = () => api.get("/dashboard/stats");
```

Screens import these functions — they do **not** embed URLs everywhere.

8.3 Typical call chain (Home dashboard)

Step	Code
Screen	`app/(tabs)/index.tsx` → `Dashboard` component
API fn	`getDashboardStats()` in `src/api/dashboard.ts`
HTTP	`GET https://…/dashboard/stats` + `Authorization: Bearer …`
Backend	`DashboardController` in `schoolsync-api`
UI	Sets `stats` state, renders cards

Errors go through `errorMessage()` in `client.ts` for user-facing text.

8.4 Helper: `useLoad`

File: **`src/lib/useLoad.ts`**

Many screens use:

```
const { data, loading, error, reload } = useLoad(() => getFoo().then(r =>
r.data.data), []);
```

Pattern: `response.data.data` is the inner payload (see JSON envelope below).

8.5 App ↔ backend endpoint map (main areas)

App module	HTTP prefix (backend)	Example
`auth.ts`	`/auth`	`POST /auth/login`
`dashboard.ts`	`/dashboard`	`GET /dashboard/stats`
`notices.ts`	`/notices`	`GET /notices`
`notifications.ts`	`/notifications`	`GET /notifications`
`attendance.ts`	`/attendance/students`	mark student attendance
`teacherAttendance.ts`	`/attendance/teachers`	staff attendance
`timetable.ts`	`/timetable`	periods / weekly view
`records.ts`, `studentRecords.ts`	`/records`, `/student/records`	assignments, exams
`registrations.ts`	`/registrations`	account approval queue
`superAdmin.ts`	`/super-admin`	cross-school admin

App module	HTTP prefix (backend)	Example
`profile.ts`	`/profile`	profile / school name

Full controllers live under `schoolsync-api/.../controller/` in the backend repo.

---

9. JSON the app sends and receives

9.1 Standard envelope (almost every response)

Backend type: `ApiResponse<T>` → `{ success, message, data }`

Success example (login complete):

```
{
  "success": true,
  "message": "OK",
  "data": {
    "status": "COMPLETE",
    "auth": {
      "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
      "userId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
      "schoolId": "8b2e1c40-1234-5678-9abc-def012345678",
      "role": "TEACHER",
      "name": "Priya Sharma",
      "schoolName": "Demo Public School"
    },
    "selectionToken": null,
    "studentOptions": null
  }
}
```

Multi-student on one username/phone:

```
{
  "success": true,
  "message": "OK",
  "data": {
    "status": "SELECT_STUDENT",
    "auth": null,
    "selectionToken": "short-lived-token-string",
    "studentOptions": [
      {
        "userId": "...",
        "name": "Arjun",
        "schoolName": "Demo School",
        "classLabel": "Class 8-A"
      }
    ]
  }
}
```

The app handles this in `extractLoginPayload()` (`src/api/loginAuth.ts`).

9.2 Login request body

```
{
  "phone": "QA-SCH-001-STU-0001",
  "otp": "your-password-here"
}
```

Field name `otp` is **the password** for password login (same as website).

9.3 After login — authorized request

```
GET /dashboard/stats HTTP/1.1
Host: schoolsync-api-production.up.railway.app
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
Accept: application/json
```

Example response:

```
{
  "success": true,
  "message": "OK",
  "data": {
    "totalStudents": 420,
    "totalClasses": 24,
    "totalTeachers": 32,
    "teachersPresent": 30,
    "teachersAbsent": 2,
    "myClassCount": 6,
    "classesAttMarked": 4,
    "myAttendancePercentage": null
  }
}
```

9.4 Error response

```
{
  "success": false,
  "message": "Invalid credentials",
  "data": null
}
```

Axios throws; `errorMessage()` prefers `err.response.data.message`.

---

10. Roles and what users see

Roles come from the JWT / `AuthResponse.role` (enum on backend), e.g.:

`STUDENT`, `TEACHER`, `PRINCIPAL`, `COORDINATOR`, `SCHOOL_ADMIN`, `SUB_COORDINATOR`, `DIRECTOR`, `CHAIRMAN`, `SUPER_ADMIN`

Mechanism	File	Behavior
Home dashboard	`app/(tabs)/index.tsx`	`SUPER_ADMIN` → `SuperAdminHome`; others → school dashboard with stats
Bottom tabs	`app/(tabs)/_layout.tsx`	Hides **Alerts** tab for `SUPER_ADMIN` (`href: null`)
Menu list	`src/menu.ts` → `menuFor(role)`	Filters `MENU` items by `roles` array

Mechanism	File	Behavior
Quick actions on home	`(tabs)/index.tsx`	Buttons depend on `role` constants (`ADMIN_ROLES`, `MARK_ATTENDANCE_ROLES`, …)

Super admin operates across schools; **school roles** are scoped to one `schoolId` in the token.

---

11. Walkthrough: login then home dashboard

Step A — Guest opens app

1. `index.tsx` → no user → `/contact`
2. User taps **Sign in** → `router.push("/login")`

Step B — Login screen (`app/login.tsx`)

1. `loginWithPassword(phone.trim(), password)`
2. `extractLoginPayload` returns `auth` object
3. `applySession(auth)` writes token + user
4. `router.replace("/(tabs)")` — replaces login in history so Back does not return to password screen

Step C — Tabs layout (`app/(tabs)/_layout.tsx`)

1. If no user → redirect `/login` (safety net)
2. Renders tab navigator: Home, Notices, Alerts (if role allows), Menu

Step D — Home (`app/(tabs)/index.tsx`)

1. `useAuth()` → `user.role`, `user.name`
2. `getDashboardStats()` in `useEffect` / `load()`
3. Renders greeting, KPI cards, `NoticeStrip`, quick actions
4. `NoticeStrip` (`src/components/NoticeStrip.tsx`) separately calls `getNotices()`

---

12. Walkthrough: any feature screen

Example: **Account requests / approvals**

Layer	Location
Entry from UI	Home quick action or Menu → path `/approvals`
Screen file	`app/approvals.tsx`
API	`src/api/registrations.ts` — `getRequests`, approve/reject helpers
Backend	`RegistrationAdminController` — `/registrations/...`

To trace a screen you do not know:

1. Search `app/` for the route filename (e.g. `approvals.tsx`).
2. Read imports from `../../src/api/...`.
3. Open that api file for the exact path and HTTP method.
4. Grep the same path in `schoolsync` backend controllers.

---

13. Configuration, builds, and environments

13.1 Local development

1. `npm install`
2. Copy `.env.example` → `.env`, set `EXPO_PUBLIC_API_BASE_URL`
3. `npx expo start` → scan QR with Expo Go (needs Metro on a PC) **or** use EAS **preview** APK (no PC)

13.2 EAS cloud build (what you use for Samsung testing)

Profile	Output	Use when
`preview` / `pilot`	**APK**	Hand to testers; uses `env.EXPO_PUBLIC_API_BASE_URL` in `eas.json`
`development`	Dev client APK	Needs Metro / dev server to load JS
`production`	**AAB**	Google Play only — not sideload friendly

See **`PILOT_RUNBOOK.md`** for Expo projectId, install QR, Android 16KB / SDK 53 notes.

13.3 What gets baked into the APK

At **build time**, `EXPO_PUBLIC_*` env vars are inlined into the JS bundle. Changing the API URL requires a **new build** (or a custom update pipeline), not just editing Railway.

14. Where the website fits in

Topic	Website (`schoolsync-web`)	Mobile app
API host	Often `/api` proxy → backend in dev	Direct `API_BASE_URL`
Login UI	`AuthShell`, same credentials	`app/login.tsx`
Navigation	Sidebar / routes in `src/lib/navigation.js`	Tabs + `src/menu.ts` (kept in sync manually)
Token storage	Browser storage / cookies pattern	`expo-secure-store`

When adding a feature, backend changes happen in **`schoolsync`** once; web and app each get a UI that calls the new endpoint.

---

15. Glossary

Word	Meaning in this project
Easool	Product name shown on phone and easool.com
SchoolSync	Internal / repo name for backend modules
EAS	Expo Application Services — cloud builds
Metro	JS bundler dev server (`expo start`)
Context	React way to share `user` without prop drilling (`AuthContext`)
Interceptor	Axios hook that runs before/after every HTTP call
Route group `(tabs)`	Folder name for tab layout without extra URL segment
Pilot / preview build	Internal APK for QA

---

16. What to read next in the code

Suggested order for a **guided tour** (open files in the IDE in this sequence):

1. `src/config.ts` — where API URL is defined
2. `src/api/client.ts` — how every HTTP call gets the token

3. `src/api/auth.ts` + `src/api/loginAuth.ts` — login contract
4. `src/auth/AuthContext.tsx` — session lifecycle
5. `app/_layout.tsx` — providers and auth gate
6. `app/index.tsx` — first redirect
7. `app/login.tsx` — sign-in UX and navigation after success
8. `app/(tabs)/_layout.tsx` — tab shell
9. `app/(tabs)/index.tsx` — home dashboard + role branching
10. `src/menu.ts` — full list of deep links by role
11. Any feature you care about under `app/` + matching `src/api/*.ts`

For **backend** login implementation (Java):

- `schoolsync-api/.../controller/AuthController.java`
- `schoolsync-api/.../service/AuthService.java`
- `schoolsync-common/.../dto/response/LoginResult.java` and `AuthResponse.java`

Quick reference: auth + API in one diagram

> **Diagram (flow chart):** The full interactive version is in `APP_GUIDE.md` under the same section. Summary:

```
flowchart TB
    subgraph Phone["Mobile app"]
        UI[app/*.tsx screens]
        CTX[AuthContext user + token]
        SEC[SecureStore ss_token / ss_user]
        AX[axios client.ts]
        UI -->…
```

Document version: aligned with Easool mobile on Expo SDK 53 (`schoolsync-app`). Update this guide when you add major navigation or auth changes.